SMARTSTAY AI

Memoria técnica del proyecto · Reto 1 · SAA + PIA + MIA

Curso 2026–2027 | Versión para revisión · 12 de noviembre de 2026

Sistema de apoyo a la decisión para estimar precios de alojamientos turísticos en España, aplicar reglas de negocio y automatizar avisos y registros mediante n8n.

Alcance: prototipo académico. No modifica automáticamente tarifas publicadas en Airbnb. Las estimaciones son orientativas y requieren criterio humano.

1. Resumen ejecutivo

SmartStay AI combina un modelo de regresión supervisada, una interfaz Streamlit, un motor de once reglas de negocio y un flujo de n8n. A partir de las características de un anuncio estima un precio por noche, lo compara con el precio anunciado y produce una decisión explicada. El modelo elegido es Random Forest general optimizado, almacenado en models/mejor_modelo.joblib. El envío de decisiones a n8n es explícito y opcional.

2. Necesidad de negocio y objetivos

Los gestores de alojamientos necesitan referencias para revisar precios y priorizar casos. El proyecto persigue: estimar precios orientativos, detectar discrepancias, justificar recomendaciones, registrar decisiones y enviar notificaciones cuando corresponda. No pretende predecir ocupación real, ingresos futuros ni determinar precios óptimos con garantías.

3. Arquitectura funcional

Datos Airbnb → limpieza y análisis exploratorio → entrenamiento y comparación → Random Forest guardado → formulario Streamlit → predicción → motor de reglas Python → decisión y motivo → envío voluntario al webhook de n8n → acciones diferenciadas.

Componentes principales: notebooks/ (EDA y experimentos), models/ (modelo serializado), app/streamlit_app.py (interfaz), src/reglas.py (decisión), src/n8n.py (comunicación HTTP), n8n/ (workflow), resultados/ (métricas y evidencias).

4. Datos y preparación

El conjunto original unificado reúne aproximadamente 109.437 anuncios y 20 columnas de nueve zonas: Madrid, Girona, Barcelona, Mallorca, Málaga, Sevilla, Valencia, Euskadi y Menorca. La variable objetivo es price, expresada en euros por noche. Se documentan valores ausentes, variables categóricas, distribución asimétrica y precios extremos.

4.1 Variables utilizadas

Tipo	Variables
Numéricas (8)	latitude, longitude, minimum_nights, number_of_reviews, reviews_per_month, calculated_host_listings_count, availability_365, number_of_reviews_ltm
Categóricas (3)	zona, room_type, estancia_larga

estancia_larga es una característica derivada de minimum_nights (30 noches o más). El pipeline incorpora imputación, StandardScaler y OneHotEncoder. La selección y las exclusiones de variables deben justificarse en el notebook de EDA.

4.2 Análisis exploratorio

El notebook SmartStay_01_EDA.ipynb documenta tipos de variables, ausencias, distribuciones, valores extremos y relaciones entre variables. Incluye gráficos descriptivos que sustentan las decisiones de preprocesamiento. El precio mediano observado inicialmente es aproximadamente 170 €, con valores máximos atípicos muy elevados.

5. Entrenamiento y selección del modelo

Se compararon regresión lineal, KNN, Random Forest, XGBoost y SVR, con un modelo Dummy como referencia. También se entrenó una MLP base y tres variantes, con validación y registro de experimentos en MLflow. Se probaron ajustes de hiperparámetros, selección de variables y un modelo especializado en alojamientos de hasta 500 €.

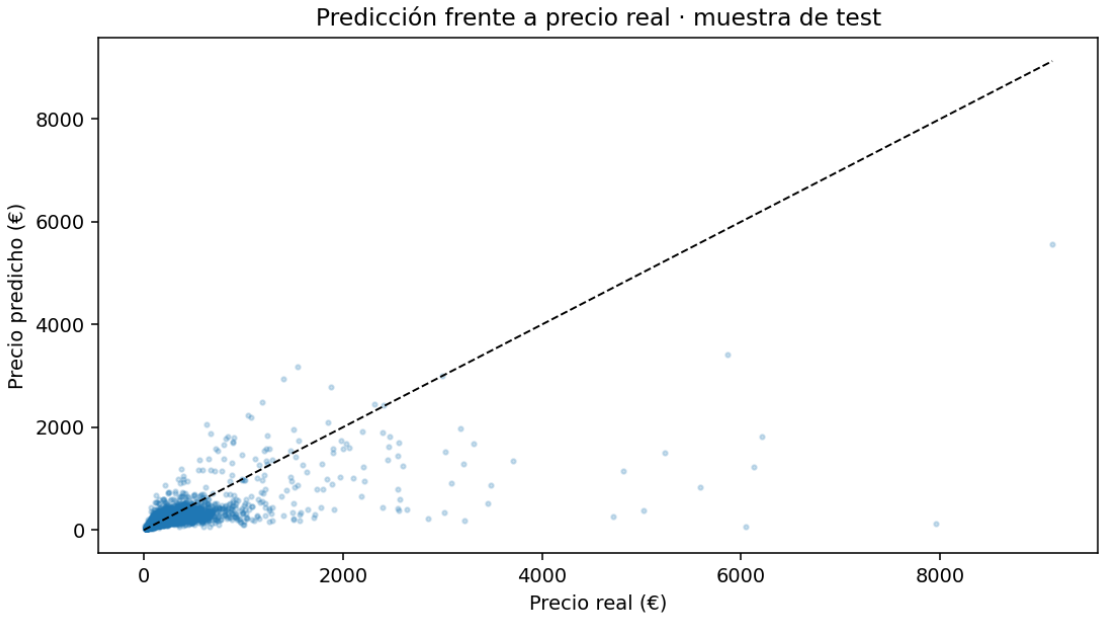
La solución seleccionada es un RandomForestRegressor general con max_features=0.7, min_samples_leaf=2, random_state=42 y n_jobs=1. Está envuelto en TransformedTargetRegressor, con log1p durante entrenamiento y expm1 en predicción. Para obtener euros debe utilizarse modelo.predict() sobre las once columnas originales.

5.1 Resultados del test reservado

Métrica	Valor	Interpretación
MAE	100.87 €	Error absoluto medio por anuncio
RMSE	322.47 €	Penaliza especialmente errores grandes
R^2	0.3973	Proporción de variabilidad explicada

La evaluación corresponde a 17.923 anuncios reservados para test. El modelo explica aproximadamente el 39,73 % de la variación del precio; su rendimiento es moderado. Los alojamientos extremos elevan el RMSE. No deben interpretarse los errores agregados como intervalos de confianza individuales.

5.2 Análisis real frente a predicho



La diagonal representa la predicción perfecta. La dispersión aumenta con el precio y se aprecian casos de subestimación en el segmento alto. El gráfico representa una muestra aleatoria reproducible de 6.000 puntos del CSV de 17.923 registros para mejorar su legibilidad.

5.3 Experimento especializado

Métrica	General · test completo	Especializado · test completo
MAE	100,87 €	115,23 €
RMSE	322,47 €	401,46 €
R^2	0,3973	0,0659

El modelo especializado mejora ciertos resultados retrospectivos en el subconjunto con precio real ≤500 €, pero empeora sobre el test completo. Dado que el precio real no se conoce para un anuncio nuevo, no se usa ese dato para elegir modelo en producción. Se mantiene el Random Forest general.

6. Explicabilidad con SHAP

El notebook de modelos incluye un análisis SHAP sobre una muestra de 50 observaciones de validación y las 23 características transformadas. Se generan representaciones de importancia y beeswarm. Entre las variables destacadas figuran minimum_nights, room_type, ubicación, coordenadas y availability_365. Las contribuciones se interpretan en escala transformada log1p y no implican causalidad.

7. Aplicación Streamlit

La aplicación app/streamlit_app.py presenta tres páginas: Inicio, Evaluación del modelo y Nueva predicción. La evaluación muestra métricas del test completo y un subconjunto de precios reales ≤500 €, así como una comparación de modelos. Estas métricas son valores documentados; la página no recalcula el test.

La nueva predicción recoge zona, tipo de alojamiento, precio anunciado, estancia mínima, reseñas, disponibilidad y actividad del anfitrión. La zona determina coordenadas urbanas aproximadas; esta simplificación puede afectar la precisión. El modelo produce el precio estimado y src/reglas.py devuelve decisión, motivo y controles de revisión. Un botón separado permite enviar el resultado a n8n.

8. Motor de reglas y seguridad

El motor implementa R1–R11. Primero valida números y rangos; después comprueba disponibilidad; a continuación clasifica el precio anunciado en relación con el estimado y considera las reseñas recientes. R11 revisa discrepancias superiores al 15 % y fuerza REVISAR PRECIO cuando superan el 30 %, salvo salidas anteriores por invalidez o falta de disponibilidad.

Condición	Decisión
Datos inválidos	REVISAR
Disponibilidad = 0	SIN DISPONIBILIDAD
Actual / estimado < 0,85	SUBIR
0,85 ≤ ratio ≤ 1,15	MANTENER
Ratio > 1,15 y reseñas recientes ≥ 10	MANTENER
Ratio > 1,15 y reseñas recientes ≤ 2	BAJAR
Ratio > 1,15 y reseñas recientes 3–9	REVISAR
Discrepancia relativa > 30 %	REVISAR PRECIO (control R11)

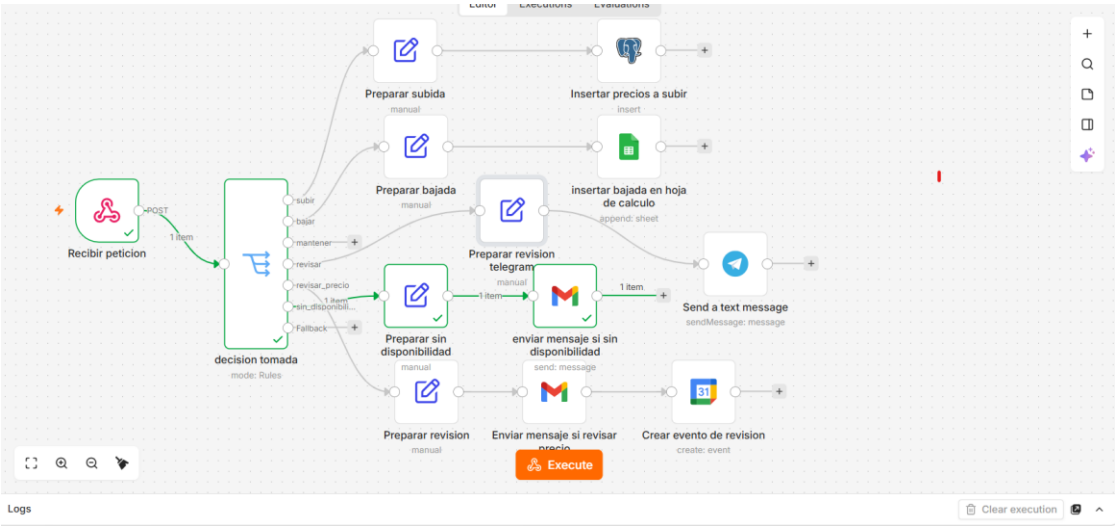
El código establece aplicar_automaticamente=False y no realiza modificaciones en Airbnb. Una discrepancia frente a la predicción no equivale a incertidumbre estadística. Las decisiones de alto impacto deben ser revisadas por una persona.

9. Automatización con n8n

src/n8n.py carga N8N_URL, N8N_CABECERA y N8N_CLAVE desde .env, envía JSON mediante requests.post con cabecera de autenticación y timeout de 10 segundos, y detecta errores HTTP. La interfaz no envía datos al calcular: requiere pulsar «Enviar decisión a n8n».

Decisión	Acción documentada
SUBIR	Insert en PostgreSQL (Neon)
BAJAR	Append Row en Google Sheets
MANTENER	Sin acción externa
REVISAR	Aviso Telegram
REVISAR PRECIO	Gmail y Google Calendar
SIN DISPONIBILIDAD	Aviso Gmail

El usuario ha confirmado pruebas reales de las seis ramas. Se dispone de capturas del workflow y de notificaciones de Telegram; la memoria específica Memoria_Tecnica_n8n_SmartStay_AI.pdf contiene más detalles. La confirmación de ejecución no sustituye una batería de pruebas automatizadas reproducibles.



10. Evidencias y reproducibilidad

Se han aportado los notebooks de EDA y modelos, código de Streamlit, reglas y envío a n8n, README, metricas_test.csv y analisis_errores_test.csv. También existen capturas de MLflow y n8n. El modelo .joblib y los datos originales deben conservarse en el proyecto, sin incluir secretos en la entrega.

Para reproducir: instalar requirements.txt en un entorno compatible; colocar los CSV en data/, el modelo en models/, configurar .env local; ejecutar python -m streamlit run app/streamlit_app.py; iniciar n8n con el workflow importado y credenciales propias; comprobar una predicción y cada decisión. Nunca subir .env ni tokens al repositorio.

11. Matriz de verificación del reto

Área	Evidencia disponible	Pendiente / comprobación
SAA	EDA, cinco familias, MLP, métricas, gráfico, SHAP	Revisar rúbrica indicador por indicador y defensa
PIA	Pipeline guardado, Streamlit, MLflow, README	Gráfico en Streamlit; prueba completa en entorno final
MIA	11 reglas, webhook, seis ramas confirmadas, capturas	Exportación JSON sin credenciales y evidencias por rama

12. Limitaciones y mejoras futuras

Precisión moderada, sesgo potencial por precios extremos, coordenadas de referencia por zona, falta de variables de equipamiento y estacionalidad, dependencia de servicios externos y ausencia de despliegue público seguro. Posibles mejoras: validación temporal, controles de calidad de datos, monitorización de deriva, trazabilidad de ejecuciones, reintentos y observabilidad de n8n.

13. Conclusiones

SmartStay AI demuestra un ciclo funcional de datos → modelo → interpretación → recomendación → automatización. Su aportación principal es combinar predicción con reglas auditables y supervisión humana. El modelo sirve como referencia orientativa, no como sistema autónomo de fijación de precios. La memoria debe acompañarse de los notebooks, código, capturas y demostración operativa (video).

Fuentes internas revisadas: README_SmartStay_AI_actualizado.md; SmartStay_01_EDA.ipynb; SmartStay_02_modelos.ipynb; app/streamlit_app.py; src/reglas.py; src/n8n.py; metricas_test.csv; analisis_errores_test.csv; Memoria_Tecnica_n8n_SmartStay_AI.pdf; capturas de n8n y MLflow.